PRESENTATION & TOOLING ARCHITECTURE

DOC ID: WC-ARCH-2026-VOL4

WildCard 게임 아키텍처 분석 보고서 (Vol. 4)

MVVM UI 시스템, 비주얼 큐 이벤트 버스 및 에디터 툴링 아키텍처

프로젝트:	WildCard (Unreal Engine 5.7 Dedicated Server)	**작성 일시:**	2026-10-19
핵심 기술:	ModelViewViewModel (MVVM), GameplayTag Visual Cue Bus, Slate Details Customization	**에디터/도구:**	WildCardEditor, SHexTileCheckBox, pipeline.py, dump_asset_metadata.py
문서 범위:	반응형 MVVM UI, Zero-Touch 메인 HUD 라우터, 계층형 전투 로그, 커스텀 슬레이트 헥사곤 맵 페인터, 자율 에이전트 파이프라인		

1. 언리얼 엔진 MVVM (Model-View-ViewModel) UI 아키텍처

1.1. MVVM 도입 배경과 아키텍처적 이점

전통적인 언리얼 엔진 UMG 개발 방식은 Event Tick 주기 바인딩이나 무거운 이벤트 그래프 스크립팅으로 인해 바이너리 충돌(Binary Merge Conflicts)과 CPU 폴링 오버헤드를 유발했습니다. WildCard는 언리얼 엔진 5.7의 **ModelViewViewModel (MVVM)** 플러그인을 전면 도입하여 이 문제를 해결하였습니다.

MVVM 계층	해당 클래스 예시	책임 및 분리 원칙
Model (모델)	InGamePlayerState, PlayerStatComponent, PlayerCardComponent	순수 게임플레이 상태 및 네트워크 복제 데이터 보관. UI의 존재를 전혀 알지 못함.
ViewModel (뷰모델)	UWildHUDViewModel, UWildCardViewModel, UMulliganViewModel	UMVVMViewModelBase 상속. 모델의 데이터를 뷰가 표시하기 좋은 형태(텍스트, 색상, 가시성)로 가공.
View (뷰)	WBP_InGameHUD, WBP_HandWidget, WBP_CardWidget	위젯 블루프린트 에셋. 뷰모델의 프로퍼티 변화(FieldNotify)를 감지하여 자동 갱신.

1.2. UWildHUDViewModel 반응형 바인딩 구현

아래는 플레이어 마나 및 게임 타이머가 변경될 때, UE_MVVM_SET_PROPERTY_VALUE 매크로를 통해 UMG 위젯으로 즉각적인 속성 변경 알림을 전달하는 뷰모델 구현체입니다.

Source/WildCard/InGame/UI/ViewModel/WildHUDViewModel.cpp

C++ / UE 5.7

```cpp
void UWildHUDViewModel::SetPlayerMana(int32 NewMana)
{
    // 이전 값과 동일하면 불필요한 Slate 리드로우를 차단하고, 변경 시에만 위젯에 필드 노티파이 전파
    UE_MVVM_SET_PROPERTY_VALUE(PlayerMana, NewMana);
}

void UWildHUDViewModel::SetRemainingTime(int32 NewTime)
{
    UE_MVVM_SET_PROPERTY_VALUE(RemainingTime, NewTime);
}

void UWildHUDViewModel::SetCurrentPhaseText(const FText& NewPhaseText)
{
    UE_MVVM_SET_PROPERTY_VALUE(CurrentPhaseText, NewPhaseText);
}
```

2. Zero-Touch Main HUD 및 비주얼 큐 (Visual Cue) 단일 이벤트 버스

2.1. 핵심 3대 설계 원칙

WildCard의 UI 및 비주얼 이펙트 연출 파이프라인은 **협업 효율성과 바이너리 충돌 방지**를 최우선으로 고려한 3대 원칙을 따릅니다.

■ Zero-Touch Visual Cue 3대 원칙

1. Zero-Touch Main HUD

메인 HUD(WBP_HUD)에는 개별 스킬 연출을 직접 코딩하지 않음. 오직 태그 매칭에 따라 연출 위젯을 띄우는 **라우터(Router)** 역할만 수행.

2. 독립 연출 위젯 (1 연출 = 1 WBP)

모든 연출 UI는 UWildVisualEffectWidget을 상속받아 독립 제작. 연출 완료 시 FinishVisualEffect()로 자가 소멸.

3. 단일 NetMulticast 통로

스킬마다 별도의 RPC를 양산하지 않고 AInGameState::Multicast_BroadcastVisualCue 단일 버스로 전파.

2.2. UWildHUDWidget의 비주얼 큐 라우팅 구현

클라이언트의 UWildHUDWidget은 AInGameState::OnVisualCueReceivedDelegate를 구독하고 있다가, 패킷이 도착하면 등록된 VisualEffectWidgetMap에서 일치하는 태그의 연출 위젯을 화면에 동적으로 생성합니다.

Source/WildCard/InGame/UI/Widget/WildHUDWidget.cpp — C++ / UE 5.7

```cpp
void UWildHUDWidget::HandleVisualCueReceived(const FWildVisualCueData& CueData)
{
    if (!CueData.CueTag.IsValid()) return;

    // 1. 등록된 맵에서 태그에 해당하는 독립 연출 위젯 클래스 탐색
    if (TSubclassOf<UWildVisualEffectWidget>* WidgetClassPtr = VisualEffectWidgetMap.Find(CueData.CueT
    {
        if (TSubclassOf<UWildVisualEffectWidget> EffectWidgetClass = *WidgetClassPtr)
        {
            // 2. 독립 위젯 인스턴스 생성 및 뷰포트 배치
            UWildVisualEffectWidget* EffectWidget = CreateWidget<UWildVisualEffectWidget>(GetOwningPla
            if (EffectWidget)
            {
                EffectWidget->AddToViewport(100); // 연출 우선순위 Z-Order 적용
                EffectWidget->PlayVisualEffect(CueData); // 위젯 애니메이션 및 사운드 재생
                UE_LOG(LogWildCard, Log, TEXT("[VisualCue Router] 연출 위젯 스폰 성공: %s"), *CueData.Cu
            }
        }
    }
}
```

2.3. UWildVisualEffectWidget의 자가 소멸 (Self-Termination) 패턴

Source/WildCard/InGame/UI/Widget/WildVisualEffectWidget.cpp C++ / UE 5.7

```cpp
void UWildVisualEffectWidget::FinishVisualEffect()
{
    // 연출 블루프린트 애니메이션 종료 이벤트(OnAnimationFinished)에서 호출됨
    OnVisualEffectFinished.Broadcast(this);

    // 뷰포트에서 스스로 제거하여 메모리 누수 방지
    RemoveFromParent();
}
```

3. 계층형 동기화 전투 로그 시스템 (WildCombatLogComponent)

WildCard는 복잡한 카드 발동과 LIFO 체인 공방의 전후 맥락을 플레이어가 한눈에 파악할 수 있도록 **계층형 트리 구조(Hierarchical Tree Logs)**의 전투 로그 아키텍처를 탑재하였습니다.

3.1. ParentLogIndex 기반 체인 인과관계 추적

모든 로그 항목은 고유한 LogIndex와, 해당 행동을 유발하거나 체인을 건 상위 행동의 ParentLogIndex를 가집니다. 이를 통해 체인으로 발동된 카운터 마법이나 유발 효과가 어떤 행동에 대한 반응이었는지를 UI에서 들여쓰기(Indentation) 및 트리 노드로 표현할 수 있습니다.

Source/WildCard/InGame/Data/BattleData.h — C++ / UE 5.7

```cpp
USTRUCT(BlueprintType)
struct WILDCARD_API FWildCombatLogEntry
{
    GENERATED_BODY()

    UPROPERTY(BlueprintReadOnly) int32 LogIndex = 0;             // 이 로그의 고유 인덱스
    UPROPERTY(BlueprintReadOnly) int32 ParentLogIndex = -1;       // 부모 로그 인덱스 (체인/반응 인과관계
    UPROPERTY(BlueprintReadOnly) EWildCombatLogType LogType;      // Gameset, Mulligan, PhasePublic, Ch
    UPROPERTY(BlueprintReadOnly) int32 InstigatorPlayerID = -1;   // 시전자 플레이어 ID
    UPROPERTY(BlueprintReadOnly) int32 TargetPlayerID = -1;       // 대상 플레이어 ID
    UPROPERTY(BlueprintReadOnly) EWildActionType ActionType;      // 세부 행동 유형
    UPROPERTY(BlueprintReadOnly) FName CardID = NAME_None;        // 사용된 카드 ID
    UPROPERTY(BlueprintReadOnly) FString CardNameStr = "";        // 로컬라이즈된 카드 명칭
    UPROPERTY(BlueprintReadOnly) int32 ChainStackCount = 0;       // 당시 체인 스택 수
    UPROPERTY(BlueprintReadOnly) int32 SourceQ = -1;              // 시전 위치 타일 좌표
    UPROPERTY(BlueprintReadOnly) int32 SourceR = -1;
    UPROPERTY(BlueprintReadOnly) int32 TargetQ = -1;              // 목표 위치 타일 좌표
    UPROPERTY(BlueprintReadOnly) int32 TargetR = -1;
    UPROPERTY(BlueprintReadOnly) FString ExtraMessage = "";       // 커스텀 설명 또는 실패 사유
};
```

4. 커스텀 에디터 모듈 (WildCardEditor) 및 슬레이트 도구

디자이너가 직관적으로 육각형 전장 맵을 기획하고 타일 배치를 편집할 수 있도록, WildCardEditor 모듈은 언리얼 에디터의 **슬레이트 디테일 커스터마이징(IDetailCustomization)**을 확장하였습니다.

■ Slate 헥사곤 맵 페인터 아키텍처

```
[ UTileLayoutDataAsset ] ──(데이터 에셋)──► [ FTileLayoutDetails ] (디테일 커스터마이징)
                                                        │
                                                        ▼
                                              [ SHexTileCheckBox ]
                                              - 육각형 모양의 슬레이트 체크박스 위젯
                                              - 마우스 클릭 시 (Q, R) 타일 활성/비활성 토글
                                              - ScopedTransaction 연동으로 Undo/Redo 완전 지원
```

4.1. FTileLayoutDetails 슬레이트 빌더 구현

Source/WildCardEditor/WildCardEditorModule.cpp — C++ / Slate

```cpp
class FTileLayoutDetails : public IDetailCustomization
{
public:
    static TSharedRef<IDetailCustomization> MakeInstance() { return MakeShared<FTileLayoutDetails>(); }

    virtual void CustomizeDetails(IDetailLayoutBuilder& DetailBuilder) override
    {
        IDetailCategoryBuilder& Category = DetailBuilder.EditCategory(
            TEXT("HexagonMapEditor"), FText::FromString(TEXT("Hexagon Map Visual Painter")));

        // 슬레이트 커스텀 위젯 행 생성
        Category.AddCustomRow(FText::FromString(TEXT("HexGrid")))
        [
            SNew(SVerticalBox)
            + SVerticalBox::Slot().AutoHeight().Padding(5)
            [
                SNew(STextBlock)
                .Text(FText::FromString(TEXT("클릭하여 육각형 타일의 활성화 여부를 직접 페인팅하세요:")))
                .Font(FAppStyle::GetFontStyle("PropertyWindow.BoldFont"))
            ]
            + SVerticalBox::Slot().AutoHeight()
            [
                // 축 좌표계(Q, R)에 따라 육각형 체크박스를 오프셋 정렬하여 배치
                CreateHexGridSlateWidget(DetailBuilder)
            ]
        ];
    }
};
```

5. 에이전트 자동화 루프 및 파이프라인 도구 (Scripts/pipeline.py)

5.1. 에셋 바이너리 리플렉션 인스펙터 (dump_asset_metadata.py)

WildCard 프로젝트는 AI 에이전트 협업 및 자동화 과정에서 발생하는 **에셋 환각(Hallucination)**을 원천 차단하기 위해 **Ground Truth Over Guessing** 원칙을 확립하였습니다. 위젯 계층 구조, 뷰모델 컨텍스트, 애니메이션 노티파이 및 슬롯 이름을 추측하지 않고, dump_asset_metadata.py를 통해 0.005초 만에 바이너리에서 정확한 메타데이터를 추출합니다.

Scripts/dump_asset_metadata.py — Python 3.11

```python
# Fast Static Binary UAsset Inspector
def inspect_uasset(file_path: Path) -> dict:
    raw_bytes = file_path.read_bytes()

    # 1. Widget Blueprint (WBP) 내부 바인딩 및 뷰모델 컨텍스트 정규식 고속 스캔
    detected_widgets = extract_regex_tokens(raw_bytes, rb'(?:WBP_|Button_|TextBlock_|Border_)[A-Za-z0-
    detected_viewmodels = extract_regex_tokens(raw_bytes, rb'[A-Za-z0-9_]+ViewModel')
    detected_bindings = extract_regex_tokens(raw_bytes, rb'(?:Get_|OnClicked_|Bind_)[A-Za-z0-9_]+')

    return {
        "asset_name": file_path.stem,
        "is_valid_uasset": True,
        "detected_widgets": list(detected_widgets),
        "detected_viewmodels": list(detected_viewmodels),
        "detected_bindings": list(detected_bindings)
    }
```

5.2. 자율 컴파일 및 테스트 파이프라인 (Scripts/pipeline.py)

Scripts/pipeline.py는 단일 CLI 명령을 통해 에셋 인스펙션(inspect), UBT를 통한 Dedicated Server 및 Editor 자동 빌드(compile), 그리고 RunUAT 무인 헤드리스 게임플레이 자동화 검증(test)을 통합 수행하는 프로젝트의 핵심 DevOps 도구입니다.

파이프라인 모드	실행 CLI 명령어	검증 대상 및 수행 작업
inspect	python Scripts/pipeline.py inspect --asset <Name>	블루프린트, 위젯, 애니메이션 바이너리 메타데이터 JSON 추출.
compile	python Scripts/pipeline.py compile	WildCardServer 및 WildCardEditor UBT 자율 빌드 및 에러 자가 치유.
test	python Scripts/pipeline.py test	헤드리스 RunUAT -test="WildCard.CardGame" 실행 및 상태 머신 단위 검증.
all	python Scripts/pipeline.py all	컴파일부터 단위 테스트까지 전체 릴리즈 파이프라인 원스톱 검증.

6. 전체 아키텍처 결론 및 최종 요약

WildCard 프로젝트는 **언리얼 엔진 5.7의 최신 프레임워크와 Dedicated Server 네트워크 보안 표준**을 결합한 모범적인 턴제 전술 카드 배틀 아키텍처를 구현하였습니다.

- **권한 분리:** 서버의 단일 진실 공급원 원칙과 엄격한 RPC 검증(WithValidation), 개인 손패 분리 복제(COND_OwnerOnly).
- **이중 상태머신:** 대규모 매치 수명주기(MatchState)와 정밀한 턴 사이클(CardPhase)의 분리를 통한 명확한 상태 제어.
- **체인 & 동시 처리:** 비밀 메인 행동 동시 제출과 역순 LIFO 서브액션 체인의 혁신적인 하이브리드 게임플레이.
- **반응형 MVVM & Zero-Touch HUD:** UMG 바이너리 충돌을 차단하고 계층형 전투 로그 및 단일 비주얼 큐 이벤트 버스를 통한 유연한 시각 연출.
- **툴링 & 자동화:** 슬레이트 기반 헥사곤 맵 페인터 및 에이전트 자율 빌드/검증 루프 확립.

> **총괄 보고서 완성:**
> 본 4부작 아키텍처 보고서는 WildCard 코드베이스의 모든 C++ 모듈, 서브시스템, 네트워크 프로토콜 및 UI 연출 체계를 빠짐없이 문서화하였으며, 향후 시스템 확장 및 라이브 서비스 운영의 공식 아키텍처 가이드북으로 활용됩니다.